ParkPulse: colored street lines feature, build prompt

Paste everything below the line into an AI coding assistant that already has the ParkPulse Flutter project (Flutter, Firebase, `flutter_map` v7). It adds the feature where each street block is drawn as a colored line showing its parking rule.

---

1. Goal

Add a **street rules layer** to the map. Each block of street that has a known parking rule is drawn as a thick colored line on the real street: **green = free**, **blue = paid or time limited**, **red = no parking**. When the user zooms in, a text label (FREE, PAID or NO PARKING) appears at the middle of each line. Tapping a label opens a small sheet with the rule text. Lines are created automatically when users report a rule or scan a sign, and can also be bulk-imported from a city's open-data file.

Important meaning: a line says what the posted rule allows on that block. It does not say a spot is open right now. Open spots remain the separate navy "P" pins.

Do not change any existing feature. Make small, careful edits to the existing `lib/main.dart`. No new packages are needed (`http`, `flutter_map`, `latlong2` and `cloud_firestore` are already installed).

2. Data model

New Firestore collection **`segments`**. One document per street block:

Field	Type	Meaning
`uid`	string	Author's uid, or `city-import` for bulk imports
`type`	string	`free`, `paid` or `no`
`text`	string, under 200 chars	Plain-English rule, for example "Up to 2 hours, 8 AM to 6 PM, Monday to Friday."
`lat`, `lng`	number	The middle point of the line. Used only to find segments in the visible map area

Field	Type	Meaning
`pts`	list of maps, 2 to 20 items	The line's points, each `{lat: number, lng: number}`, in drawing order. Firestore cannot store a list inside a list, so each point is a map
`createdAt`	server timestamp	When it was created

Security rules to add inside `match /documents`

```
match /segments/{id} {
  allow read: if request.auth != null;
  allow create: if request.auth != null
    && request.resource.data.uid == request.auth.uid
    &&
request.resource.data.keys().hasOnly(['uid','type','text','lat','lng','pts','
createdAt'])
    && request.resource.data.pts.size() >= 2 &&
request.resource.data.pts.size() <= 20
    && request.resource.data.text.size() < 200;
}
```

Clients can only create, never update or delete. Bulk imports use the Firebase Admin SDK, which bypasses the rules.

3. Changes to `lib/main.dart`

3.1 Imports and model class

Add `import 'dart:async';` at the top. Add this top-level class next to the existing `typedef Doc`:

```
class Seg {
  final String id, type, text;
  final List<LatLng> pts;
  Seg(this.id, this.type, this.text, this.pts);
  LatLng get mid => pts[pts.length ~/ 2];
}
```

3.2 New state in `_MapPageState`

- `List<Seg> _segs = [];` the segments currently loaded.
- `double _zoom = 15;` the current map zoom, used for label and pin switching.
- `Timer? _deb;` a debounce timer.

The existing `_filter` (null, `free`, `paid`, `garage`) and the `_col` color map are reused. `_col` must hold: `free` green (`Colors.green`), `paid` `Color(0xFF1E6FD9)`, `no` `Colors.red`.

3.3 `_moved()`: load segments for the visible area

1. Cancel any running `_deb`, then start a new `Timer` of **500 ms** so loading only happens after the map stops moving.
2. When it fires, read `_map.camera`. If the widget is mounted, `setState` to save `_zoom = camera.zoom`.
3. If the zoom is **below 15**, stop (do not query).
4. Get `camera.visibleBounds`. Query `segments` with `where('lat', isGreaterThan: bounds.south)` and `where('lat', isLessThan: bounds.north)`, `limit(300)`, using `.get()`. This is a range on one field only, so it needs no composite index.
5. Firestore can only do one range filter, so filter longitude in code: skip any document whose `lng` is less than `bounds.west` or greater than `bounds.east`.
6. Convert every kept document into a `Seg`, converting each point map into `LatLng` with `(num).toDouble()` casts.
7. If still mounted, `setState(() => _segs = result)`.

Call `_moved` in two places: pass `onMapReady: _moved` and `onPositionChanged: (_, __) => _moved()` in the existing `MapOptions`.

3.4 `_makeSegment(LatLng ll, String type, String text)`: create a line from a report

This snaps a reported rule onto the real street using free OpenStreetMap data.

1. Build this Overpass query string (use raw strings so the dollar sign is not interpreted): `[out:json][timeout:10];way(around:25,LAT,LNG)[highway~"^(residential|tertiary|secondary|primary|unclassified|living_street|service)$"];out geom;` where LAT and LNG are the reported point. `around:25` means within 25 metres.

2. Send `GET https://overpass-api.de/api/interpreter?data=<url-encoded query>` with header `User-Agent: ParkPulse/1.0`.
3. Parse `elements`. Each element has a `geometry` list of `{lat, lon}` points (note the key is `lon`).
4. Over all elements, find the single point (vertex) nearest to the reported location, using `Distance().as(LengthUnit.Meter, ...)`. Remember which way (element) and which index `bi` it belongs to.
5. If nothing was found, return silently.
6. Take the line piece from index `bi - 1` to `bi + 1` inclusive (clamped to the list ends). That is about one block.
7. Compute the middle point as `pts[pts.length ~/ 2]`.
8. Add a document to `segments` with `uid`, `type`, `text`, `lat`, `lng` (the middle point), `pts` (list of `{lat, lng}` maps) and `createdAt: FieldValue.serverTimestamp()`.
9. Call `_moved()` so the new line shows up.
10. Wrap the whole function in try/catch and ignore errors, because the map pin from the old flow already exists as a fallback.

Call `_makeSegment(ll, type, t.text.trim())` at the end of the existing `_addRule` method, right after the "Rule added. +5 points" message. This means long-press rules, the Report Rule button and the sign scanner all create lines, because they all go through `_addRule`.

3.5 Drawing the lines

Inside the `FlutterMap` children, directly after the `TileLayer` and before the existing marker layers, add:

1. A `PolylineLayer` with one `Polyline` per segment: `strokeWidth: 6`, color from `_col[type]`, points from the segment. Show a segment only if `_filter == null` or `_filter == s.type`. The Garages filter therefore hides all lines.
2. A `MarkerLayer` of labels, only when `_zoom >= 16`. One `Marker` per visible segment at `s.mid`, `width: 90`, `height: 24`. Child: `GestureDetector(onTap: () => _segSheet(s))` wrapping a centered `Text`.
   - Text is `FREE`, `PAID` or `NO PARKING`.
   - Style: color `_col[type]`, `fontWeight: FontWeight.w800`, `fontSize: 12`, and a white text shadow (`Shadow(color: Colors.white, blurRadius: 3)`) so it stays readable over the map.

3. Change the existing rule-pin markers (the circles from the `rules` collection) so they only appear when `_zoom < 16`. At zoom 16 and above the lines and labels replace them.

The open-spot pins, garage markers and destination marker stay unchanged and stay above the lines.

3.6 `_segSheet(Seg s)`

A modal bottom sheet with padding 20 containing: a title in bold size 20 colored by type ("Free parking", "Paid or time limit" or "No parking"), the rule `text` at size 16, and grey helper text "Reported by the community. Always check the posted sign."

4. Bulk import from city open data (`tools/import_city_parking.py`)

A Python script run by the developer, not part of the app.

- Usage: `python import_city_parking.py regulations.geojson serviceAccount.json`.
- Install `firebase-admin`. Initialize with the service-account key.
- Include a docstring telling the user to download a parking-regulation GeoJSON from their city's open-data portal and check its licence.
- A `classify(props)` function the user edits. The default joins all property values into one uppercase string: `NO PARKING` or `NO STOPPING` gives `('no', 'No parking (city data)')`; `METER`, `PAID`, `HR` or `HOUR` gives `('paid', 'Paid or time limited (city data)')`; anything else gives `('free', 'No restriction found (city data)')`.
- Read `LineString` and `MultiLineString` features. Truncate each line to 20 points, skip lines shorter than 2 points. GeoJSON order is `[lng, lat]`, so swap when writing.
- Write documents with `uid: 'city-import'`, the middle point as `lat` and `lng`, `pts` as `{lat, lng}` maps, and a server timestamp.
- Commit in batches of 400 and print how many segments were imported.

5. Acceptance tests

1. Zoom out below 15: no lines are drawn and no segment queries run.

2. Report a "No parking" rule on a real street. Within a few seconds a red line appears along that block of the street and the rule pin disappears once you zoom to 16.
3. Zoom to 16 or more: the text label appears in the middle of the line and tapping it opens the sheet with the rule text.
4. Tap the Free chip: only green lines remain. Tap Garages: all lines disappear.
5. Pan the map by two blocks: new lines for that area load after the map stops moving, and lines outside the screen are dropped.
6. Report a rule where there is no road (a field or water): no line is created and no error is shown.
7. Run the import script on a small test GeoJSON: lines show up in the app for that area.
8. Two phones: a line created on one appears on the other after panning.

6. Known limits to write in the README

- A line is drawn on the street's centre, not on the correct curb side.
- The rule text is stored, but the color does not change with the time of day (a 7 to 9 AM no-parking rule stays red all day). Time-aware coloring is a later step: decode the rule at draw time.
- Each report makes one block-sized line, so coverage only grows as people report.
- The public Overpass and OpenStreetMap services are for light use. For launch, use a paid or self-hosted service.
- The viewport query reads up to 300 segments, and can miss some in very dense areas.
- Rules from users can be wrong. Voting (below) reduces this.

7. Optional next step: voting

Add `up` and `down` counters to each segment, with buttons "Still correct" and "Wrong" in the segment sheet. Each user can vote once per segment (store votes in `segments/{id}/votes/{uid}`). A Cloud Function updates the counters. Draw a dashed or faded line when a segment has fewer than 2 up-votes or more down-votes than up-votes, and hide it after 5 net down-votes.

8. Do not

- Do not copy data from other parking apps. Use city open data and your own users' reports.
- Do not claim a line means a spot is available.
- Do not break existing features: spot pins, garage markers, alerts, scanning and points must work exactly as before.